# Guide to the Practical Sessions

**Reinforcement Learning Course**

Universidad de Alcalá

October 14, 2026

## Contents

**1 Introduction** **2**

**2 Course Structure** **2**

**3 What is a Jupyter Notebook (`.ipynb`)?** **2**

**4 Accessing the Notebooks (Blackboard & GitHub)** **2**

**5 Opening a Notebook: Two Options** **3**
5.1 Option A: Google Colab (recommended) . . . . . 3
5.2 Option B: Working Locally with an IDE . . . . . 3

**6 Working on the Notebook** **4**

**7 Saving Your Work** **5**

**8 Class Competition: Automated GitHub Leaderboard** **5**

**9 Tournament Evaluation Criteria and Optimization Strategies** **5**
9.1 Evaluation Criteria: What is Rewarded? . . . . . 6
9.2 Optimization Strategies: How to Achieve Higher Scores? . . . . . 6

**10 Head-to-Head Agent Duel: Comparing Against Classmates** **7**

**11 Submitting Your Practical** **8**

**12 Evaluation Overview** **8**

**13 Getting Help** **9**

## 1 Introduction

This document explains how the practical sessions of the Reinforcement Learning (RL) course are organized, and how to work with the provided materials. These materials are distributed as **Jupyter Notebooks** (`.ipynb` files). The purpose of this guide is to help every student get started, even without any prior experience using notebooks or Google Colab.

## 2 Course Structure

The course combines theory sessions (9h) and practical sessions (6h). The practical part consists of **three mandatory deliverables**, each based on a Jupyter Notebook:

| Session | Topic | Notebook |
|---|---|---|
| Practical 1 | Introduction to RL & Deep PPO | `Practical1/unit1_uah_student.ipynb` |
| Practical 2 | Tabular Q-Learning (Taxi-v3/v4) | `Practical2/unit2_uah_student.ipynb` |
| Final Practical | Final project | `Practical_final/unit3_uah_student.ipynb` |

In every practical session, students will: interact with RL environments, train models following a guided (Colab-based) tutorial, implement the requested parts of the code, and submit their final solution. **Practical 1** and **Practical 2** are evaluated and contribute to the **Practical Mark (PM)** of the course. The **Final Practical** is not part of the PM: it will instead be assessed as part of the **oral exam**, through its defense and oral questions.

## 3 What is a Jupyter Notebook (`.ipynb`)?

A Jupyter Notebook is a document that mixes executable code (in this course, Python) with explanatory text, images, and the results of execution, all in the same file. It is organized into *cells*, of two main types:

- **Code cells**: contain Python code that can be executed by clicking the Play button or pressing `Shift + Enter`.
- **Text cells (Markdown)**: contain explanations, instructions, equations and images, but no executable code.

You do **not** need to install Python or any library on your own computer to work with these notebooks: the recommended option is **Google Colab**, a free cloud platform provided by Google that runs notebooks directly in your web browser. However, if you prefer to work **locally** on your own computer using an IDE (e.g. VS Code, PyCharm or Jupyter itself), this is also possible and explained in this guide.

## 4 Accessing the Notebooks (Blackboard & GitHub)

The `.ipynb` files for the practical sessions **are available both on Blackboard** (the Universidad de Alcalá's e-learning platform) and directly in the official **course GitHub repository** (https://github.com/gramuah/RL_tournament).

**Step by step**

1. Navigate to the course page on Blackboard and locate the section for the corresponding practical session (Practical 1, Practical 2, or Final Practical).
2. Click on the `.ipynb` file to download it (e.g. `unit1_uah_student.ipynb`). Depending on your browser, it may download directly or open a preview with a download button.
3. Save the file somewhere easy to find on your computer, such as the *Downloads* folder or Desktop.

**Important**

Do **not** try to open the `.ipynb` file by double-clicking it directly on your computer: it will not display correctly, since it requires a Jupyter/Colab environment to be rendered and executed. Follow the steps in the next section instead.

## 5 Opening a Notebook: Two Options

Once you have the `.ipynb` file downloaded from Blackboard, you can choose one of the following two options to work with it. **Option A (Google Colab)** is recommended for most students, since it requires no installation and runs in the cloud. **Option B (Local IDE)** is for students who prefer to work on their own computer, already have Python installed, or want full control over their environment.

### 5.1 Option A: Google Colab (recommended)

**Step by step**

1. Go to `https://colab.research.google.com`.
2. Sign in with a Google account (any personal Gmail account works).
3. In the welcome window, click on the tab **Upload**. If it does not appear, go to **File** → **Upload notebook**.
4. Click **Browse** (or drag and drop the file) and select the `.ipynb` file you downloaded from Blackboard.
5. The notebook will open directly in Colab, ready to be executed.

Advantages of this option: no installation required, free access to GPUs, and your work can be saved directly in your Google Drive (see Section 7).

### 5.2 Option B: Working Locally with an IDE

If you prefer to run the notebook on your own machine, you will need to install Python and a few additional tools. This option is suitable if you already have a Python environment set up, or if you want to avoid depending on an internet connection.

**Step by step**

1. **Install Python** (version 3.9 or higher is recommended) from https://www.python.org/downloads/, if not already installed.
2. **Install an IDE with notebook support**, for example:
   - **VS Code**: https://code.visualstudio.com/, plus the official *Python* and *Jupyter* extensions (installable from the Extensions tab inside VS Code).
   - **PyCharm Professional**: https://www.jetbrains.com/pycharm/, which has built-in Jupyter notebook support.
   - Alternatively, install **Jupyter Notebook/JupyterLab** directly: open a terminal and run `pip install notebook`, then launch it with `jupyter notebook`.
3. **Open the `.ipynb` file** you downloaded from Blackboard using your chosen IDE (e.g. **File** → **Open** in VS Code/PyCharm, or navigating to it from the Jupyter file browser).
4. **Select a Python interpreter/kernel** for the notebook (VS Code and PyCharm will prompt you to choose one the first time you open a notebook).
5. **Install the required libraries**. Most notebooks include a first code cell with `!pip install ...` commands; run that cell first. If needed, you can also install packages manually from a terminal with `pip install` *`package-name`*.

**Important**

When working locally, your computer's hardware (CPU/GPU) will be used for training, which may be significantly slower than Colab's free GPUs if your machine does not have a dedicated graphics card. It is also your responsibility to manage library versions and dependencies correctly. If you encounter installation issues, Option A (Google Colab) remains available as a fallback.

## 6 Working on the Notebook

Once the notebook is open, either in Colab or in your local IDE, keep the following in mind:

- **Run cells in order**, from top to bottom, using the Play button to the left of each cell (or `Shift + Enter`). Skipping cells or running them out of order can cause errors.
- **Read the text cells carefully**: they contain the explanations and instructions needed to understand and complete the exercises.
- **Complete the sections marked for you**, usually indicated with comments such as `# TODO` or similar placeholders, following the given instructions.
- **(Colab only) Enable a GPU** when training models, since it speeds up computation considerably: go to **Runtime** → **Change runtime type** → **Hardware accelerator** → **GPU**, then save. If working locally, make sure your own GPU drivers (e.g. CUDA) are correctly configured, or expect slower training on CPU.
- If a cell throws an error, re-read the previous cells: you may be missing a required installation step (e.g. `!pip install ...`) or a variable defined earlier that was not executed.

## 7 Saving Your Work

If you are using **Google Colab**, it does **not** save your changes back to the original file automatically. To avoid losing your progress:

1. As soon as you open the notebook, go to **File** → **Save a copy in Drive**. This creates your own editable copy in *My Drive* → *Colab Notebooks*.
2. Continue working exclusively on that copy. From this point on, Colab will save your changes automatically in your Google Drive.
3. You can reopen your copy anytime from `https://colab.research.google.com` under the **Recent** tab, or directly from Google Drive.

If you are working **locally** with an IDE, simply save the file regularly using the standard shortcut (`Ctrl+S`), as you would with any other document, and keep a backup copy of your work (e.g. in a personal folder or cloud storage) in case of hardware issues.

## 8 Class Competition: Automated GitHub Leaderboard

In **Practical 1** and **Practical 2**, the final section of the notebook will guide you through the packaging of your agent: an `agent.py` file (which defines an `agent(observation)` function) together with your trained model weights (`ppo-LunarLander-v3.zip` or `q-learning.pkl`).

To foster friendly competition, peer learning, and benchmark reproducibility, all agents participate in an **automated class tournament hosted directly on GitHub**:

1. **Submission via GitHub Pull Request**: You can submit directly from the notebook! The final section of each practical includes a 1-click cell that automatically forks the repository, uploads your files, and creates the Pull Request for you using your GitHub token. Alternatively, you can do it manually by placing your files in:
   - Practical 1: `submissions/practical1/<your_github_username>/`
   - Practical 2: `submissions/practical2/<your_github_username>/`

   and opening a **Pull Request (PR)** against the main repository.
2. **Instant CI Evaluation**: A **GitHub Actions** robot runner immediately triggers, runs your agent across fixed, fair benchmark seeds, and posts a detailed evaluation card (mean reward, standard deviation, success rate, and inference latency) directly as a comment on your PR within 2 minutes!
3. **Official Leaderboard**: Once your PR is approved and merged, your agent is automatically ranked on the official `LEADERBOARD.md` at the root of the repository, competing for podium rankings (1st, 2nd, 3rd place).

**Leaderboard Tip**

You can submit multiple times! If you improve your hyperparameters, refine reward engineering, or discover a better architecture, open an updated PR to improve your standing on the leaderboard.

## 9 Tournament Evaluation Criteria and Optimization Strategies

To excel in the tournament leaderboard and win head-to-head duels, it is essential to understand both **how the automated evaluation engine scores agents** and **which engineering techniques yield the highest performance**.

### 9.1 Evaluation Criteria: What is Rewarded?

When you open a Pull Request, the automated CI runner evaluates your agent under rigorous, identical benchmark conditions:

1. **Mean Reward ($\mu$, Primary Ranking Metric)**: The average score achieved across 30 standardized deterministic seeds $(42, 43, 44, \ldots)$.
   - **LunarLander-v2/v3**: Moving from top to landing pad scores +100 to +140 pts; each leg touching the surface scores +10 pts; touching down between flags and cutting engines earns +100 pts bonus; crashing penalizes −100 pts; firing the main thruster costs −0.3 pts/frame, and side thrusters cost −0.03 pts/frame.
   - **Taxi-v3/v4**: Successfully delivering the passenger at the destination awards +20 pts; each navigation step costs −1 pt (penalizing inefficient meandering); attempting an illegal pickup or drop-off action incurs a severe −10 pts penalty.
2. **Success Rate (%)**: Percentage of episodes completed without crash or timeout. A competitive agent must achieve a 100% success rate.
3. **Stability and Low Variance ($\pm\sigma$)**: Evaluates policy robustness against varying terrain, wind turbulence, and randomized passenger locations. Lower $\sigma$ ranks higher in tie-breaks.
4. **Speed and Step Efficiency (Steps)**: In head-to-head duels and in Taxi-v3, taking the minimal number of time steps (shortest Manhattan trajectory) directly breaks score ties and decides the winner.
5. **Inference Latency (ms)**: Average forward-pass execution time per step. Policies must answer in under 100 ms per step to guarantee real-time deployment.

### 9.2 Optimization Strategies: How to Achieve Higher Scores?

**For Practical 1 (LunarLander Deep PPO):**

- **Network Architecture**: Increase the default MLP capacity from $[64, 64]$ to $[128, 128]$ or $[256, 256]$ hidden units with orthogonal weight initialization (`ortho_init=True`) and `Tanh` or `ReLU` activations.
- **Hyperparameter Tuning**:
  - Set the discount factor $\gamma = 0.999$ and GAE $\lambda = 0.98$ to account for long-horizon landings.
  - Use learning rate decay (starting at $3 \times 10^{-4}$ and decaying linearly or with a cosine schedule).
  - Tune the entropy coefficient (`ent_coef` $\in [0.001, 0.01]$) to encourage initial exploration while guaranteeing stable convergence.
  - Set PPO clipping parameter to $\epsilon = 0.2$.
- **Vectorized Parallel Training**: Use `make_vec_env("LunarLander-v2", n_envs=8)` to decorrelate batch transitions and stabilize gradient updates.
- **Observation Normalization**: Apply `VecNormalize(norm_obs=True, norm_reward=False)` so that altitude, velocities, and orientation angles are scaled evenly.
- **Fuel Economy**: Once your agent learns to land safely, refine the policy so it minimizes unnecessary lateral thruster burns, squeezing an additional +20 to +30 points toward the top tier ($> 280$ pts).

- **Deterministic Evaluation**: Always set `deterministic=True` during evaluation and when exporting your submission.

**For Practical 2 (Taxi Tabular Q-Learning):**

- **Exponential Epsilon Decay**: Use a smooth exponential schedule ($\epsilon_t = \epsilon_{\min} + (\epsilon_{\max} - \epsilon_{\min}) \exp(-\text{decay} \times t)$) with decay $\approx 0.0005$ over 25,000–30,000 episodes. This ensures all 500 discrete state-action pairs are visited adequately before freezing the policy.
- **Learning Rate Decay**: Start with $\alpha = 0.7$ for rapid early updates, and gradually decay $\alpha$ toward 0.1 so the Q-table converges smoothly without oscillations.
- **Discount Factor** $\gamma \in [0.95, 0.99]$: Prioritizes immediate, shortest-path actions over long-horizon wandering.
- **Strict Greedy Policy at Test Time**: In evaluation, set $\epsilon = 0$ (`action = np.argmax(Q[state])`) to completely eliminate random moves.
- **Zero-Penalty Target**: An optimal Taxi policy requires zero illegal pickups/dropoffs ($-10$) and takes between 11 and 14 steps, achieving a mean score between $+7.5$ and $+9.5$ points per episode.

## 10 Head-to-Head Agent Duel: Comparing Against Classmates

Beyond overall leaderboard ranks, students can directly challenge and compare their agents against baselines or other classmates in a **Head-to-Head Duel**:

- **Identical Benchmark Seeds**: Both agents face the exact same initial state, landing terrain, wind turbulence (in LunarLander), or passenger pickup/drop-off locations (in Taxi).
- **Speed vs. Score Analysis**: Discover who lands or delivers faster (fewer time steps), who consumes less fuel/actions, and who achieves higher reward stability.
- **Side-by-Side Synchronized Video**: Generates a broadcast-style split-screen video with a real-time HUD (speed, step counter, accumulated reward, touchdown status) and celebration victory screens.
- **Challenge Any Classmate**: By entering the GitHub username of any student on the leaderboard, the comparator tool automatically pulls their model from the public repository and runs the duel against your local model.

### How to View and Run Duels Directly on GitHub

You do not need to install anything locally or run Python to visualize duels:

1. **Interactive Web Arena (GitHub Pages)**: Visit the official public web arena at https://gramuah.github.io/RL_tournament/. You can pick any two participants from dropdown menus, click **"Iniciar Duelo"**, and watch the split-screen replay with live landing speed gauges, fuel burn rates, and comparative statistics.
2. **Animated GIFs in** `LEADERBOARD.md`:
   The repository's official leaderboard (see `LEADERBOARD.md`) includes embedded animated duel replays showing the agents maneuvering and landing side-by-side directly inside GitHub Markdown.

3. **On-Demand Duels via GitHub Actions**: In the **Actions** tab of the repository (see `Actions → duel.yml`), click **"Run workflow"**, select your agent and an opponent, and the CI bot will execute the match and post an animated GIF replay with full scorecards directly in the job summary.

4. **Inside Google Colab**: Run the dedicated duel cell provided at the end of each practical notebook to execute duels and render the synchronized video inline.

## 11 Submitting Your Practical

At the end of each practical session (or by the deadline indicated by the instructor), you must submit your work for evaluation. To do so:

1. Make sure **all cells have been executed** and their outputs are visible. Do not clear the outputs before submitting: the instructor needs to see the results of your training/experiments.

2. Download/export your notebook as an `.ipynb` file:

   - **Google Colab**: **File** → **Download** → **Download .ipynb**.
   - **Local IDE**: the file on your disk *is* already the `.ipynb` file; just make sure it has been saved with the latest changes (`Ctrl+S`).

3. Rename the file following the convention:
   `Surname_Name_PracticalX.ipynb`
   (e.g. `Garcia_Juan_Practical1.ipynb`).

4. **For Practical 1 and Practical 2**, also download the `submission.zip` file generated at the end of the notebook (see Section 8).

5. Submit both the notebook and, when applicable, the `submission.zip` file through the **Blackboard** assignment/submission box corresponding to that practical session, before the deadline indicated by the instructor.

**Important**

All three deliverables (**Practical 1**, **Practical 2** and the **Final Practical**) are mandatory and must be submitted. Practical 1 and Practical 2 are evaluated individually and contribute to the Practical Mark (PM) of the final grade. The Final Practical will be assessed during the oral exam, through its defense and oral questions. Late or missing submissions may negatively affect your final mark.

## 12 Evaluation Overview

As a reminder, the final mark of the course is composed as follows:

- **Practical Mark (PM)**: based on **Practical 1** and **Practical 2**, described in this guide.
- **Oral Exam**: accounts for **70%** of the final grade, and includes the defense of the **Final Practical**.

**Originality and Research Bonus**

In all practical assignments, originality, critical thinking, and independent research are actively valued and rewarded. Students who go beyond the baseline solutions—by thoroughly experimenting with hyperparameters, exploring alternative algorithmic strategies or neural network architectures, and clearly documenting their investigations—will be rewarded with higher marks.

## 13 Getting Help

If you encounter errors or have doubts while working on a notebook:

- Re-read the instructions in the notebook: most exercises include hints or links to relevant documentation.
- Check the original Hugging Face Deep RL Course material on which these practicals are based:
  - Unit 1: https://huggingface.co/learn/deep-rl-course/en/unit1/hands-on
  - Unit 2: https://huggingface.co/learn/deep-rl-course/en/unit2/hands-on
  - Unit 3: https://huggingface.co/learn/deep-rl-course/en/unit3/hands-on
- Ask the instructor during the practical sessions, or by email, if the problem persists.